CPNA v1 — Agentic RAG Clinical Pediatric Nutrition Assistant

AI Product Management Case Study

Role: Technical AI PM — zero to shipped MVP **Stack:** Python · FastAPI · Next.js 14 · Qdrant · Redis · HuggingFace DistilBERT · Anthropic Claude API **Status:** MVP shipped. Active development ongoing.

Executive Summary

CPNA v1 is a clinical-grade pediatric nutrition assistant I built from scratch as a Technical AI PM. It uses a multi-agent RAG pipeline to support four query types : personalized therapy planning, dietary recommendations, food comparisons, and general nutrition education, with deterministic safety guardrails that ensure no clinical output is ever generated by an LLM without a traceable, rules-based computation underneath it.

The core product insight was this: the problem in clinical nutrition AI is not missing knowledge. It is missing structure. Existing tools either ignore the patient's clinical picture entirely, or use language models to generate answers that sound authoritative but have no deterministic basis. For a child with cystic fibrosis or chronic kidney disease, that distinction is the difference between helpful guidance and a safety event.

I owned every consequential decision: the choice of agentic RAG over a simpler LLM pipeline, the rule that no nutrient target may be LLM-generated, the gatekeeper logic that prevents partial patient data from producing a plan, and the scope constraints that made v1 shippable without compromising safety. I also developed the build methodology — a 12-prompt Claude Code Prompt Pack that translated architecture into zero-ambiguity developer instructions, enabling a complex multi-agent system to be delivered with full integration from day one.

The MVP is shipped. Active development continues on the knowledge graph layer, confidence calibration, and observability dashboards.

Problem Statement

Pediatric clinical nutrition is one of the most underserved domains in health technology. Children with chronic or metabolic conditions — cystic fibrosis, PKU, Type 1 diabetes, chronic kidney disease, epilepsy requiring ketogenic therapy, and others — need individualized nutrition plans that account for their diagnosis, age, weight, medications, and biomarkers. Those plans must reference clinical guidelines, not population averages. They must adjust for drug-nutrient interactions. They must suggest foods that are actually available where the family lives.

No consumer tool does this. And the AI tools that have attempted it have failed in a predictable way: they generate responses that feel clinical but are not. A language model asked "what should a 6-year-old with CKD eat?" will produce a confident, well-formatted answer drawn from its training distribution. That answer has no binding to DRI tables, no awareness of GFR-stage protein restrictions, no knowledge of which foods to avoid based on the child's current medications. It is pattern-matching dressed as clinical reasoning.

The downstream consequence is not just inaccuracy. It is misplaced trust. A caregiver who receives a confident-sounding nutrition plan from an AI tool has no way of knowing whether that plan was computed or confabulated. In a healthy adult context, that gap is tolerable. In a pediatric clinical context, it is not.

The problem I set out to solve was not "how do we make AI answer nutrition questions better." It was "how do we build a system where every clinical output is traceable, every gap is surfaced honestly, and the LLM is never trusted to do the work that belongs to deterministic computation."

Where It Started

I didn't come to this project with a brief. I came with a frustration.

Clinical pediatric nutrition sits at one of the most consequential intersections in healthcare: the child is small, the stakes are high, the conditions are rare, and the guidance is scattered across academic papers, DRI tables, and specialist knowledge that most families and even many practitioners can't easily access or apply. A parent managing a child's cystic fibrosis, PKU, or chronic kidney disease and trying to make daily food decisions is not well-served by a generic wellness app or a Google search.

I saw a gap that existing tools — including early LLM-based nutrition assistants — weren't filling. Not because the knowledge didn't exist, but because no product had structured it correctly. Most tools either ignored clinical context entirely, or used language models to generate answers that *sounded* authoritative but had no traceable basis. That distinction mattered enormously to me. A hallucinated nutrient target for a child with CKD isn't a bad answer. It's a harmful one.

So I started building.

The Hardest PM Work Happened Before Any Code

I initiated CPNA from scratch — no inherited codebase, no existing team, no prior product to extend. That meant the first and most consequential decisions were mine alone.

Looking back, four decisions shaped everything else. I made all of them under uncertainty, and I'd make them the same way again.

Decision 1: RAG over a simpler LLM pipeline

The easy path was a prompt-engineered LLM. Fast to prototype, easy to demo, and — in a wellness context — probably fine. But this wasn't wellness. It was clinical support.

The core issue with a pure LLM for this domain is that a language model cannot be trusted to hold clinical precision at generation time. It can approximate. It can sound right. But when I'm asking it to determine energy targets for a 7-year-old with cystic fibrosis, "sounds right" is the wrong bar. The answer needs to come from DRI tables and ESPGHAN consensus guidelines, not from a model's interpolation of its training distribution.

I chose a full agentic RAG architecture not because it was more impressive, but because it was the only structure that let me separate what the LLM should do (synthesize, explain, converse) from what it should never do (compute clinical targets). That boundary became the product's primary safety guarantee.

The practical cost was real: more infrastructure, more moving parts, more to specify and test. I accepted that cost. In a clinical domain, the simpler system that fails unsafely is not the pragmatic choice — it's just the faster way to build something I couldn't stand behind.

Decision 2: No LLM for nutrient computation

This followed from the first decision, but it was a sharper line to hold. It would have been tempting, mid-build, to let the model "fill in" for edge cases the deterministic engine didn't cover yet. I didn't allow it.

Every therapy output in CPNA traces to one of two sources: the deterministic nutrient engine (which wraps validated computation logic and applies condition-specific adjustment rules from cited clinical guidelines), or the DRI tables. If neither can produce a value, the system does not produce a value. It downgrades to a recommendation and tells the user why.

The rule sounds simple. Holding it was harder. It meant the therapy gatekeeper had to be strict — the system will not compute a personalized plan if required patient data is missing, even

partially. It meant building a slot-filling flow patient enough to collect age, sex, weight, height, diagnosis, and medication data before proceeding. It meant accepting that some queries would not get the answer the user wanted, because giving them a wrong answer would be worse.

That is a product decision, not just a technical one. It defines what the product is willing to be.

Decision 3: Designing the safety rules

The safety architecture in CPNA isn't a compliance layer bolted on at the end. It's load-bearing.

I designed it around specific failure modes I identified before a line of code was written. What happens if intent classification is ambiguous? (Low-confidence results must trigger clarification, never silent routing.) What happens if the user follows a therapy session with a food comparison question, does the system silently carry the patient's data into the new query? (No. Context inheritance is explicit, not automatic.) What happens if internal orchestration fields, debug traces, computation objects, make it into the API response? (A hard error at the display adapter, not a soft warning.)

Each rule came from a specific harm I was trying to prevent. The therapy gatekeeper exists because a plan built on incomplete patient data is worse than no plan. The display adapter exists because internal fields in a UI response erode trust and reveal system internals. The context inheritance rules exist because a child's medical profile should not silently attach itself to queries where the user didn't intend that.

Designing these rules was some of the most important product work I did on this project. None of it shows up in a demo. All of it shapes whether the product is safe to use.

Decision 4: Scoping v1 ruthlessly

I made an early list of everything CPNA would not do in v1: no EMR integration, no clinician permissioning, no autonomous treatment, no role-based UX, no button-driven interaction flows.

The temptation in a zero-to-one build is to scope generously. Every deferred feature feels like a gap. But every feature added to v1 is a feature that makes the core harder to get right, and in a clinical product, getting the core right is the only thing that matters.

The no-buttons rule is the one I get asked about most. The entire workflow advances through natural text input. No CTAs, no action menus, no structured choice prompts. This wasn't minimalism for its own sake. It was a decision to eliminate a class of UX dependency bugs, to keep the state model clean, and to force the system to handle the full range of natural language input rather than routing around it with UI scaffolding. If CPNA can't handle "she's 8" as a slot

fill, that's a product problem — not something I should paper over with a button that says "Enter Age."

How I Built It: The Claude Code Prompt Pack

One of the core challenges of building a system this complex without a large engineering team is translation fidelity. The distance between what I understood the architecture to be and what a developer would implement from a standard PRD was a real risk. Specifications get interpreted. Ambiguities become decisions made without me in the room.

What I'm Proud Of

The Nigeria-specific food mapping. The system doesn't recommend salmon and fortified breakfast cereals to a family in Lagos. For users with `country="NG"`, the food source engine surfaces liver, crayfish, moringa, uziza — ingredients available in Nigerian markets and meaningful to Nigerian caregivers. This was a deliberate product decision, not a configuration detail. The user's reality is a constraint, not an edge case.

The downgrade path. When the system can't deliver therapy — because the diagnosis isn't in v1 scope, or because required data is missing — it doesn't fail silently or return an error. It downgrades gracefully to a recommendation, explains why, and tells the user what would make a full plan possible. That exit behavior is product design. It's the difference between a system that fails and one that degrades intelligently.

The observability design. I specified the logging schema to emit only derived metadata — intent, confidence, latency, downgrade flag. Never patient data. Never message text. Never biomarkers. The constraint wasn't a compliance checkbox; it was a product value decision. A system that gains operational insight at the cost of patient privacy is not a system I'd build.

What's Still Being Built

The MVP is shipped. The following are active:

The knowledge graph layer — scaffolded but not fully implemented. Case study retrieval, connecting a current patient profile to analogous documented cases in the knowledge base, is one of the highest-value capabilities the system could offer. It's the next major build.

Confidence calibration. The 0.70 intent classification threshold was set from judgment, not from a labeled evaluation dataset. The right threshold should be derived empirically, from precision-recall analysis on real query distributions. That study is in the backlog.

The observability metrics I designed are instrumented but not yet connected to a monitoring dashboard. I want intent confidence distribution and downgrade frequency surfaced in real time — these are leading indicators of model behavior drift, and in a clinical product, I want to see them before users do.

What This Project Taught Me

Building CPNA clarified something I'd suspected but not yet tested at this scale: the most important work an AI PM does is not writing user stories or running sprints. It's making the calls that determine what the system is allowed to do and what it is not — and then designing a technical architecture that enforces those calls structurally, not just aspirationally.

Every safety rule in CPNA could have been a guideline in a README. Instead, each one is a constraint in the code — a raised exception, a failing test, a hard boundary at the API layer. That's the difference between a product that behaves well in demos and one that behaves well in production.

The other thing I learned: scope discipline compounds. Every feature I said no to in v1 is something I can build correctly in v2, with a stable foundation under it. The temptation to ship everything is a form of impatience dressed up as ambition. The job is to ship the right thing well.

CPNA isn't finished. But what exists is coherent, safe, and honest about what it doesn't yet do. That's the standard I hold my work to.

Judith | *Technical AI Product Manager*